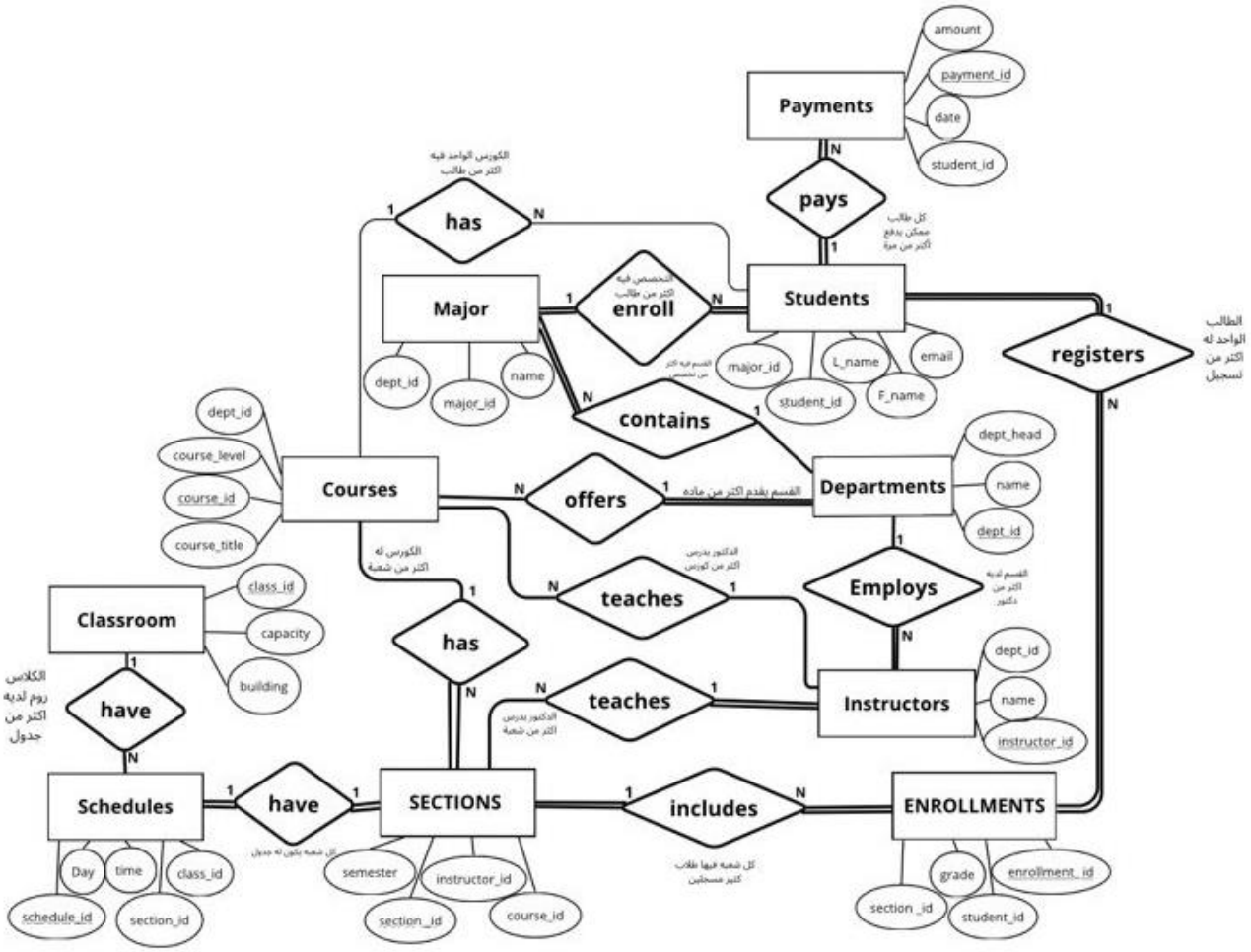
Organization Description

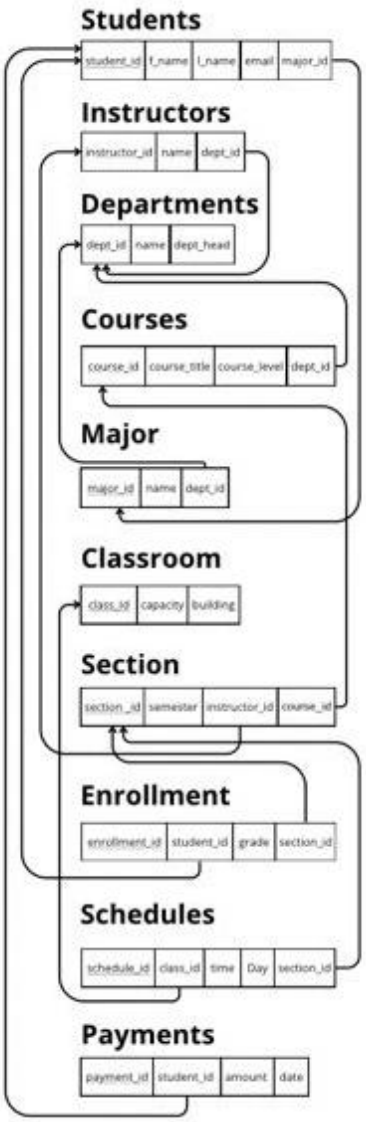
A University Management System used to store and organize data of students, instructors, departments, courses, and enrollments. It helps manage academic operations such as course registration in an organized and accurate database system. These are the 10 entities that have been implemented in this project :

- Students
- Instructors
- Departments
- Courses
- Majors
- Classrooms
- Section
- Enrollments
- Schedules
- Payments

Entity Relationship Diagram (ER Diagram)



Relational Schema Diagram / Table Diagram



Relational Schema

Students (students_ID, l_name ,f_name , email, major_id FK)

Instructors (instructor_id PK, name, dept_id FK)

Departments (dept_id PK, dept_name, dept_head)

Courses (course_id PK, course_title, course_level, dept_id FK)

Major (major_id PK, name, dept_id FK)

Classroom (class_id PK, capacity, building)

Sections (section_id PK, semester, instructor_id FK, course_id FK)

Enrollments (enrollment_id PK, student_id FK, section_id FK, grade)

Schedules (schedule_id PK, section_id FK, class_id FK, time, day)

Payments (payment_id PK, student_id FK, amount, date)

Normalization

All tables are normalized up to Third Normal Form (3NF):

- 1NF: All attributes contain atomic values
- 2NF: All attributes depend on the primary key
- 3NF: No transitive dependencies exist

Implementation (MySQL)

```
USE university;

CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL,
    dept_head VARCHAR(100)
);

CREATE TABLE Classroom (
    class_id VARCHAR(50) PRIMARY KEY,
    capacity INT,
    building VARCHAR(100)
);

CREATE TABLE Major (
    major_id INT PRIMARY KEY,
    name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Courses (
    course_id VARCHAR(10) PRIMARY KEY,
    course_title VARCHAR(100) NOT NULL,
    course_level VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Instructors (
    instructor_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
```

```
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    F_name VARCHAR(50) NOT NULL,
    L_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    major_id INT NOT NULL,
    FOREIGN KEY (major_id) REFERENCES Major(major_id)
);

CREATE TABLE SECTIONS (
    section_id VARCHAR(10) PRIMARY KEY,
    semester VARCHAR(20) NOT NULL,
    instructor_id INT,
    course_id VARCHAR(10),
    FOREIGN KEY (instructor_id) REFERENCES Instructors(instructor_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

CREATE TABLE ENROLLMENTS (
    enrollment_id VARCHAR(10) PRIMARY KEY,
    grade VARCHAR(2),
    section_id VARCHAR(10),
    student_id INT,
    FOREIGN KEY (section_id) REFERENCES SECTIONS(section_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);

CREATE TABLE SCHEDULES (
    schedule_id VARCHAR(10) PRIMARY KEY,
    section_id VARCHAR(10),
    class_id VARCHAR(50),
    time VARCHAR(50),
    day VARCHAR(20),
    FOREIGN KEY (section_id) REFERENCES SECTIONS(section_id),
    FOREIGN KEY (class_id) REFERENCES Classroom(class_id)
);

CREATE TABLE PAYMENTS (
```

```
    payment_id VARCHAR(10) PRIMARY KEY,
    student_id INT,
    amount DECIMAL(10,2),
    date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);

INSERT INTO Departments (dept_id, dept_name, dept_head) VALUES
(101, 'Computer Science', 'Dr. Faisal Al-Zahrani'),
(102, 'Information Systems', 'Dr. Nora Mansour'),
(103, 'Software Engineering', 'Dr. Khalid Omar'),
(104, 'Cybersecurity', 'Dr. Sarah Ahmed');

INSERT INTO Classroom (class_id, capacity, building) VALUES
('284', 40, 'A'),
('173', 30, 'B'),
('209', 25, 'C'),
('156', 50, 'B');

INSERT INTO Major (major_id, name, dept_id) VALUES
(47291, 'Computer Science', 101),
(80563, 'Mathematics', 102),
(11928, 'English', 103),
(66407, 'Physics', 104);

INSERT INTO Courses (course_id, course_title, course_level, dept_id) VALUES
('CS101', 'Introduction to Programming', 'Level 3', 101),
('DE202', 'Database Systems', 'Level 4', 101),
('IS301', 'Systems Analysis', 'Level 5', 102),
('SE401', 'Software Architecture', 'Level 7', 103);

INSERT INTO Instructors (name, dept_id) VALUES
('Dr. Ahood Alzahrani', 101),
('Dr. Raghda Alqurashi', 104),
('Dr. Mashael Alhajdali', 103),
('Dr. Saleh Alghamdi', 102),
('Dr. Khalid Alshammari', 103),
('Dr. Ghada Alfatni', 104);

INSERT INTO Students (student_id, F_name, L_name, email, major_id) VALUES
(445001, 'Hala', 'Albushri', 's445001@uqu.edu.sa', 47291),
(445002, 'Noura', 'Alsaadi', 's445002@uqu.edu.sa', 80563),
```

(446001, 'Khalid', 'Alotaibi', 's446001@uqu.edu.sa', 47291),
(446002, 'Ali', 'Alqahtani', 's446002@uqu.edu.sa', 11928),
(446003, 'Faisal', 'Alzahrani', 's446003@uqu.edu.sa', 66407),
(446004, 'Saad', 'Alshammari', 's446004@uqu.edu.sa', 66407),
(447001, 'Dana', 'Alotaibi', 's447001@uqu.edu.sa', 80563);

INSERT INTO SECTIONS (section_id, semester, instructor_id, course_id) VALUES
('S001', 'Fall 2026', 1, 'CS101'),
('S002', 'Fall 2026', 3, 'DE202'),
('S003', 'Spring 2026', 4, 'IS301'),
('S004', 'Spring 2026', 5, 'SE401');

INSERT INTO ENROLLMENTS (enrollment_id, student_id, section_id, grade) VALUES
('E001', 445001, 'S001', 'A'),
('E002', 445002, 'S001', 'B+'),
('E003', 446001, 'S002', 'A-'),
('E004', 446002, 'S003', 'B'),
('E005', 446003, 'S004', 'A');
INSERT INTO SCHEDULES (schedule_id, section_id, class_id, time, day) VALUES
('S01', 'S001', '284', '08:00 AM', 'Sunday'),
('S02', 'S001', '173', '08:00 AM', 'Tuesday'),
('S03', 'S002', '209', '10:00 AM', 'Monday'),
('S04', 'S003', '156', '01:00 PM', 'Wednesday');
INSERT INTO PAYMENTS (payment_id, student_id, amount, date) VALUES
('P_101', 445001, 1500.00, '2026-04-01'),
('P_102', 445002, 2000.50, '2026-04-05'),
('P_103', 446001, 1200.00, '2026-04-10');

Screen Shot

```
1 •  SELECT * FROM uni.departments;
```

Result Grid | Filter Rows: | Edit:

dept_id	dept_name	dept_head
101	Computer Science	Dr. Faisal Al-Zahrani
102	Information Systems	Dr. Nora Mansour
103	Software Engineering	Dr. Khalid Omar
104	Cybersecurity	Dr. Sarah Ahmed
NULL	NULL	NULL

```
1 •  SELECT * FROM uni.classroom;
```

Result Grid | Filter Rows: | Edit:

class_id	capacity	building
156	50	B
173	30	B
209	25	C
284	40	A
NULL	NULL	NULL

```sql
SELECT * FROM uni.enrollments;
```

Result Grid | Filter Rows: | Edit:

enrollment_id	grade	section_id	student_id
E001	A	S001	445001
E002	B+	S001	445002
E003	A-	S002	446001
E004	B	S003	446002
E005	A	S004	446003
NULL	NULL	NULL	NULL

```sql
SELECT * FROM uni.instructors;
```

Result Grid | Filter Rows: | Edit:

instructor_id	name	dept_id
1	Dr. Ahood Alzahrani	101
2	Dr. Raghda Alqurashi	104
3	Dr. Mashael Alhajdali	103
4	Dr. Saleh Alghamdi	102
5	Dr. Khalid Alshammari	103
6	Dr. Ghada Alfatni	104
NULL	NULL	NULL

Limit to 1000 rows

```sql
SELECT * FROM university.students;
```

Result Grid | Filter Rows: | Edit:

student_id	F_name	L_name	email	major_id
445001	Hala	Albushri	s445001@uqu.edu.sa	47291
445002	Noura	Alsaadi	s445002@uqu.edu.sa	80563
446001	Khalid	Alotaibi	s446001@uqu.edu.sa	47291
446002	Ali	Alqahtani	s446002@uqu.edu.sa	11928
446003	Faisal	Alzahrani	s446003@uqu.edu.sa	66407
446004	Saad	Alshammari	s446004@uqu.edu.sa	66407
447001	Dana	Alotaibi	s447001@uqu.edu.sa	80563
NULL	NULL	NULL	NULL	NULL

```sql
SELECT * FROM university.payments;
```

Result Grid | Filter Rows: | Edit:

payment_id	student_id	amount	date
P_101	445001	1500.00	2026-04-01
P_102	445002	2000.50	2026-04-05
P_103	446001	1200.00	2026-04-10
NULL	NULL	NULL	NULL

```sql
SELECT * FROM uni.courses;
```

Result Grid | Filter Rows: | Edit:

course_id	course_title	course_level	dept_id
CS101	Introduction to Programming	Level 3	101
DE202	Database Systems	Level 4	101
IS301	Systems Analysis	Level 5	102
SE401	Software Architecture	Level 7	103
NULL	NULL	NULL	NULL

```sql
SELECT * FROM uni.major;
```

Result Grid | Filter Rows: | Edit:

major_id	name	dept_id
11928	English	103
47291	Computer Science	101
66407	Physics	104
80563	Mathematics	102
NULL	NULL	NULL

```
SELECT * FROM university.schedules;
```

schedule_id	section_id	class_id	time	day
S01	S001	284	08:00 AM	Sunday
S02	S001	173	08:00 AM	Tuesday
S03	S002	209	10:00 AM	Monday
S04	S003	156	01:00 PM	Wednesday

```
SELECT * FROM university.sections;
```

section_id	semester	instructor_id	course_id
S001	Fall 2026	1	CS101
S002	Fall 2026	3	DE202
S003	Spring 2026	4	IS301
S004	Spring 2026	5	SE401

Queries

Query 1: Courses and Department Details

```
SELECT Courses.course_title, Departments.dept_name, Departments.dept_head
FROM Courses
JOIN Departments ON Courses.dept_id = Departments.dept_id;
```

course_title	dept_name	dept_head
Introduction to Programming	Computer Science	Dr. Faisal Al-Zahrani
Database Systems	Computer Science	Dr. Faisal Al-Zahrani
Systems Analysis	Information Systems	Dr. Nora Mansour
Software Architecture	Software Engineering	Dr. Khalid Omar

Query 2: Detailed Weekly Schedule

```
SELECT
    SCHEDULES.day,
    SCHEDULES.time,
    Courses.course_title,
    SCHEDULES.class_id,
    Classroom.building
FROM SCHEDULES
JOIN SECTIONS ON SCHEDULES.section_id = SECTIONS.section_id
JOIN Courses ON SECTIONS.course_id = Courses.course_id
JOIN Classroom ON SCHEDULES.class_id = Classroom.class_id
ORDER BY SCHEDULES.day;
```

day	time	course_title	class_id	building
Monday	10:00 AM	Database Systems	209	C
Sunday	08:00 AM	Introduction to Programming	284	A
Tuesday	08:00 AM	Introduction to Programming	173	B
Wednesday	01:00 PM	Systems Analysis	156	B

Query 3: Students details (only students with payments and that are enrolled will show)

```
use uni;
SELECT i.grade, s.f_name, p.amount
from enrollments i
join students s
on i.student_id =s.student_id
join payments p
on p.student_id = s.student_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

grade	f_name	amount
A	Hala	1500.00
B+	Noura	2000.50
A-	Khalid	1200.00

Query 4: Finding the hardest course

```
use uni;
select course_title as hardest_course from courses  where course_level= "level 7" ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

hardest_course
Software Architecture

Query 5: Display each major and the number of students in it

```
use university;
SELECT m.name AS Major_Name,
COUNT(s.student_id) AS Total_Students
FROM Major m
JOIN Students s ON m.major_id = s.major_id
GROUP BY m.name
ORDER BY Total_Students DESC;
```

Result Grid | Filter Rows: | Export: | Wr

Major_Name	Total_Students
Computer Science	2
Physics	2
Mathematics	2
English	1

Query 6 : Display classrooms with capacity greater than 30

```
10 •  SELECT
11    c.class_id AS Class,
12    c.building AS Building,
13    c.capacity AS Capacity
14    FROM Classroom c
15    WHERE c.capacity > 30;
```

Result Grid | Filter Rows: | Export:

Class	Building	Capacity
156	B	50
284	A	40

Query 7: Display Students Names with Their Majors

```
79 •  SELECT Students.F_name, Major.name AS Major_Name
80    FROM Students
81    JOIN Major ON Students.major_id = Major.major_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

F_name	Major_Name
Ali	English
Hala	Computer Science
Khalid	Computer Science
Faisal	Physics
Saad	Physics
Noura	Mathematics
Dana	Mathematics

Query 8: Display Students Names, Majors, and Grades for Students Who Got an A

```
83 •  SELECT
84        s.F_name AS Student_Name,
85        m.name AS Major_Name,
86        e.grade
87    FROM Students s
88    JOIN Major m ON s.major_id = m.major_id
89    JOIN ENROLLMENTS e ON s.student_id = e.student_id
90    WHERE s.student_id IN (
91        SELECT student_id
92        FROM ENROLLMENTS
93        WHERE grade = 'A');
```

Result Grid | Filter Rows: | Export:

Student_Name	Major_Name	grade
Hala	Computer Science	A
Faisal	Physics	A

Conclusion

This project successfully designed and implemented a University Management System database that organizes and stores essential academic data in a structured way. It includes 10 related tables that represent students, instructors, departments, courses, and enrollments, ensuring efficient data management and reduced redundancy. The system demonstrates the application of database design principles, normalization, and SQL implementation to support university operations in a clear and reliable manner.